

Wireless Protocol Development Internship Task

TDMA Schedule Planner and Optimizer with EMANE Integration

Project documentation: design process, thought process, day-wise work log and final values

Prepared for	Vaan Megam Networks Private Limited
Submitted by	Candidate
Work period	1 October 2026 - 7 October 2026 (7 days)
Part 1 (mandatory)	Python schedule optimizer (the "Brain")
Part 2 (bonus)	EMANE 1.5.3 TDMA integration (the "Engine")
Repository	see README.md at the repository root

Contents

1. Executive summary
 2. Requirements and where each one is met
 3. Day-wise work log (Day 1 to Day 7)
 4. Final design
 5. Results and final values
 6. EMANE integration (Part 2)
 7. Limitations and future work
 8. Reproducing the results
 9. Tools, assistance and references
- Appendix A. Full program output for the 16-node grid

1. Executive summary

The task is to plan a TDMA schedule for a wireless network so that radios sharing one frequency never collide, while radios far enough apart reuse the same time slot. The solution has two parts.

Part 1 is a Python command-line optimizer. It reads node coordinates as a JSON string, links radios that are within 500 m, builds the distance-2 (two-hop) conflict graph, and colours it with a staged heuristic pipeline. Colours are time slots. The output is the node-to-slot mapping and the Slot x Node boolean matrix in the format shown in the brief. An independent checker re-verifies every schedule from the raw coordinates.

Part 2 converts the Part 1 schedule into EMANE's TDMA schedule XML and the surrounding model profiles, and runs 16 emulated radios in one Docker container. The emulation was executed; results are in section 6.

Final values

Measure	Value
Frame length, 4x4 grid (300 m spacing), 16 nodes	9 slots (clique lower bound 9: provably optimal)
Frame length, random 16-node layout	10 slots (lower bound 10: provably optimal)
Frame length, two clusters plus isolated node	8 slots (lower bound 8: provably optimal)
Collisions in verified schedules	0 (independent checker, 40 random topologies plus examples)
Automated tests	29 passing
EMANE: node pairs behaving per the 500 m topology (10 ms slots)	120 of 120
EMANE: mean loss under 16 simultaneous flows, this schedule	0.8 %
EMANE: same load, plain distance-1 colouring	43.1 %

2. Requirements and where each one is met

Requirement from the brief	Where it is met
Python environment with networkx; radios as nodes, links as edges; 500 m range	tdma_planner/topology.py: build_connectivity_graph (inclusive 500 m, configurable with --range)
Distance-2 colouring: adjacent or sharing a neighbour must differ	topology.build_conflict_graph and tdma_planner/coloring.py
Handle direct-link and hidden-terminal cases	Conflicts are tagged direct or hidden; unit tests cover both; independent checker reports which one
Spatial reuse beyond two hops	Follows from the model; 3 hops apart share a slot (test), grid slot 0 carries 4 radios
Heuristics to reduce slot count (NP-hard problem)	DSATUR, largest-first, smallest-last, randomised restarts, TabuCol, exact branch-and-bound, clique lower bound
CLI taking 16 coordinates as a JSON string	schedule_optimizer.py --nodes '{...}' (also --nodes-file and stdin)
Slot x Node boolean matrix and node-to-slot mapping, ASCII	tdma_planner/report.py; format follows the sample in the brief
EMANE on one Linux machine or one Docker container	emane/Dockerfile (EMANE 1.5.3, Ubuntu 22.04)
TDMA model XML profiles, 1 ms slots, frames, frequencies	emane/bridge.py writes tdmamac.xml, phy.xml, nem, platform, transport; default slot 1000 us
Bridge converting the schedule matrix into an EMANE schedule event	bridge.py writes tdmastchedule.xml; loaded with emaneevent-tdmastchedule
EMANE passes or drops packets according to the schedule	Run on EMANE 1.5.3; results in section 6
Documentation: design process, thought process, final values	This report; README.md; docs/DESIGN.md; docs/EMANE_INTEGRATION.md
Git repository with source, documentation (PDF), presentation (PPT)	Repository root; docs/TDMA_Planner_Report.pdf; docs/TDMA_Planner_Presentation.pptx

3. Day-wise work log

The task had a seven-day window. The log below records what was done on each day, the reasoning behind it, and what it produced.

Day 1 - 1 October: understanding the problem and fixing the model

Goal. Understand TDMA, interference, and what EMANE expects, and settle the mathematical model before writing code.

Work done

- Read the brief and the referenced material: EMANE TDMA model guide, NetworkX tutorial, and papers on distance-2 colouring for wireless networks.
- Wrote the three scenarios out by hand: two radios in range, a hidden terminal (A - B - C), and two radios three hops apart.
- Worked out by hand what the schedule must guarantee for a 3-node and 4-node chain, to have expected answers for later tests.

Thought process and decisions

- Interference is modelled as a unit-disk graph: edge when distance ≤ 500 m. The brief says "within 500 meters", so the boundary is inclusive.
- A slot assignment is a proper colouring of the square of the connectivity graph (G^2). That one idea gives collision avoidance and spatial reuse together, so spatial reuse needs no separate algorithm.
- Decided early to compute a lower bound (largest clique) so that quality can be reported with evidence instead of being claimed.

Outcome. A written model and a list of test cases; the repository skeleton.

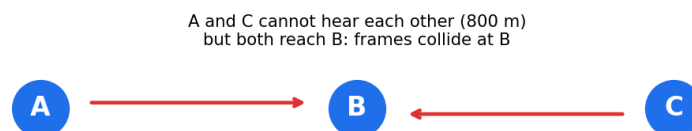


Figure 1. The hidden-terminal case. A and C are 800 m apart and cannot hear each other, but both are heard by B, so they must not share a slot.

Day 2 - 2 October: graph layer and input handling

Goal. Turn coordinates into graphs, with strict input handling.

Work done

- Wrote topology.py: JSON parsing, validation, connectivity graph, and distance-2 conflict graph.
- Rejected malformed input explicitly: bad JSON, empty object, wrong arity, non-numeric values, NaN and Infinity, each with a clear message and exit code 2.
- Sorted node names naturally, so Node_2 precedes Node_10.

Thought process and decisions

- The conflict graph is built from neighbour sets, not from an all-pairs shortest-path computation, and records whether each conflict is direct or hidden. That makes failures explainable later.
- Accepted 2-D or 3-D coordinates; the brief shows 2-D but the model does not depend on it.

Outcome. Graph layer finished; first unit tests for direct and hidden-terminal conflicts.

Day 3 - 3 October: constructive heuristics, CLI and report

Goal. Get a first correct, end-to-end schedule and the report format from the brief.

Work done

- Implemented DSATUR, largest-first and smallest-last greedy colouring, and randomised DSATUR restarts.
- Wrote report.py to reproduce the sample layout: header block, node-to-slot list, and the Slot x Node boolean matrix. Added a slot-to-node view that makes spatial reuse visible.
- Wrote cli.py with --nodes, --nodes-file, stdin, --range, --seed, --restarts and --json-out.

Thought process and decisions

- DSATUR first because it is the standard strong constructive method for colouring and is deterministic with a fixed tie-break.
- Each stage keeps the best result found so far, so later stages can only improve the answer.
- Seeded randomness (--seed) so every run is reproducible, which matters for a demo and for debugging.

Outcome. A working command-line tool producing the required output.

Day 4 - 4 October: reducing the slot count, and proving correctness

Goal. Push the heuristics beyond greedy, and make the correctness claim independent of the optimizer.

Work done

- Added TabuCol local search: fix $k = \text{best} - 1$ colours and move conflicting nodes with a tabu list until no conflicts remain.
- Added an exact DSATUR branch-and-bound for small graphs, which either finds a better colouring or proves the current one optimal. Added the clique lower bound.
- Wrote verify.py: a checker that works only from coordinates and does not share code with the graph builder or the colouring.
- Wrote 24 tests, including 40 random topologies checked by the independent verifier.

Thought process and decisions

- The verifier is deliberately separate: if the optimizer and its checker shared a bug, they would agree with each other and still be wrong.
- The report states whether the frame length is provably optimal or only lies in an interval [lower bound, result]. It never claims more than was proved.
- A 200-node random field colours in well under a second, so the pipeline is not limited to 16 nodes.

Outcome. Part 1 complete: all three 16-node examples reach their clique lower bound.

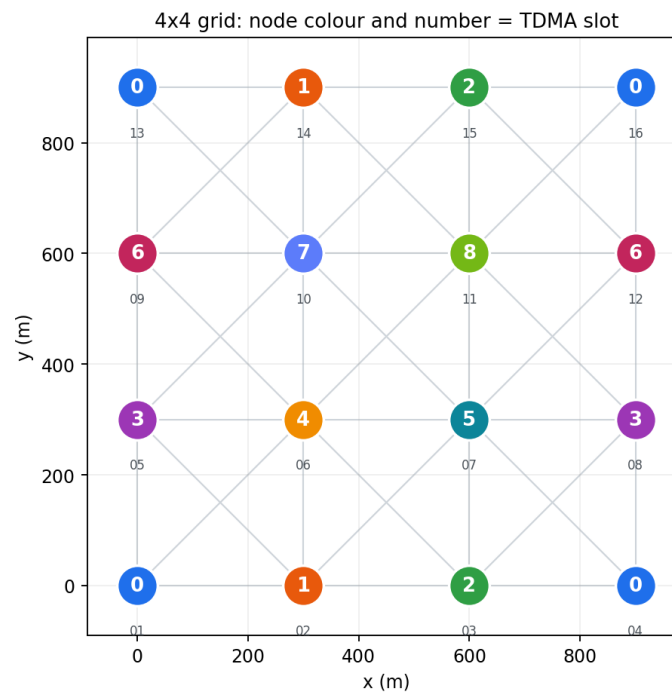


Figure 2. The 4x4 grid after optimisation. Same colour means same slot: nodes 1, 4, 13 and 16 transmit together in slot 0 because they are more than two hops apart.

Day 5 - 5 October: EMANE research and the bridge

Goal. Learn exactly what EMANE's TDMA model consumes, and build the converter.

Work done

- Studied the EMANE TDMA schedule format: structure (frames, slots, slot duration), multiframe (frequency, datarate), frame, and slot elements with tx and rx.
- Wrote bridge.py. Each node transmits in its coloured slot and listens in every other slot. Nodes sharing a colour appear in one nodes list.
- Added a pathloss event file so EMANE reproduces the 500 m disk model exactly instead of approximating it with free-space RF numbers.
- Wrote slot_sim.py: a slot-by-slot replay (a frame is decoded only if exactly one transmitter is audible), usable without EMANE.

Thought process and decisions

- The bridge reads the Part 1 JSON and never recomputes anything, and it refuses to export a schedule that breaks the two-hop rule.
- The brief asks for 1 ms slots; that is the default (slotduration 1000 microseconds).
- In the simulator the distance-2 schedule delivers all 84 directed links; a plain distance-1 colouring loses 64 frames to hidden-terminal collisions.

Outcome. Bridge output validated for well-formedness and against the Python schedule in tests.

Day 6 - 6 October: running EMANE and fixing what it rejected

Goal. Run the generated configuration on a real EMANE and make it work.

Work done

- Built a Docker image with EMANE 1.5.3 on Ubuntu 22.04. One network namespace per radio, one emane process each, an event service for the pathloss events, and emaneevent-tdmaschedule for the schedule.
- EMANE rejected the first attempt several times. Fixed: missing subid in the PHY profile, -Inf not accepted as an EEL timestamp, wrong flag for the pid file, event service started from the wrong directory, and a missing networkx package in the image.
- Wrote check_schedule.sh (pings every node pair) and load_test.sh (16 simultaneous flows), and a baseline mode that runs the same test with a plain distance-1 colouring.

Thought process and decisions

- Documentation-only configuration was not enough; every one of the failures above was found only by running it.
- At the specified 1 ms slots, sixteen non-realtime processes on a laptop miss slot boundaries (92 of 120 pair checks pass). At 10 ms all 120 pass, because only the timing margin changes, not the schedule logic. Both numbers are reported.
- The comparison against a naive colouring is the key test: it shows that the extra slots buy correctness.

Outcome. EMANE enforced the schedule: 120 of 120 pairs correct, 0.8 % loss under load (43.1 % for the naive schedule).

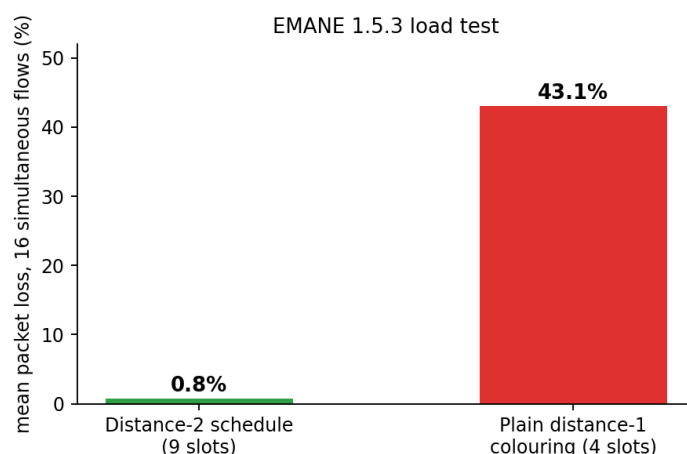


Figure 3. Mean packet loss with 16 simultaneous flows on EMANE 1.5.3 (10 ms slots).

Day 7 - 7 October: documentation, presentation, packaging

Goal. Finish the deliverables the brief lists: source code, documentation as PDF, presentation as PPT, in a Git repository.

Work done

- Wrote README, design notes and this report; built the slide deck for the presentation and demo.
- Re-ran all 29 tests, regenerated the sample outputs, and reviewed every deliverable against the brief (section 2).

Thought process and decisions

- The documentation is organised so a reader can follow the reasoning, not only the results.
- Limitations are stated openly in section 7.

Outcome. Repository ready for submission.

4. Final design

4.1 Model

Connectivity graph G . One vertex per radio; an edge joins two radios whose distance is at most the radio range (500 m by default). **Conflict graph $C = G$ squared.** Two radios conflict if they are one hop apart (direct interference) or two hops apart (hidden terminal). A slot assignment is a proper colouring of C .

4.2 Optimisation pipeline

Stage	Purpose
Clique lower bound	Any clique in C needs that many slots; gives a target and a proof of optimality when matched.
DSATUR, largest-first, smallest-last	Fast constructive heuristics; keep the best.
Randomised DSATUR (300 restarts)	Escape unlucky tie-breaks.
TabuCol	Try to remove one more colour by local search.
Exact branch-and-bound (up to 60 nodes)	Find a better colouring or prove the current one optimal.

4.3 Verification

verify.py checks every pair of nodes in the same slot straight from coordinates: are they in range of each other, or do they share a neighbour in range? The CLI exits non-zero and names the offending pair if the check fails. The message "Schedule verified conflict-free" is printed only after this check passes.

4.4 Interface between the parts

schedule_optimizer.py --json-out writes the schedule (positions, node-to-slot, matrix). bridge.py reads only that file, so Part 2 contains no scheduling logic.

5. Results and final values

Topology	Links	Frame length	Lower bound	Status
grid_16 (4x4, 300 m)	42	9	9	provably optimal
random_16 (1200 m square)	39	10	10	provably optimal
clusters_16 (two clusters + one isolated node)	47	8	8	provably optimal

In the grid case five of the nine slots carry two or more simultaneous transmitters; slot 0 carries four.

Slot-level replay (84 directed links, 4x4 grid)

Schedule	Slots	Delivered	Collided
Distance-2 schedule (this project)	9	84	0
Plain distance-1 colouring	4	20	64
Random assignment, same slot count	8	31	39

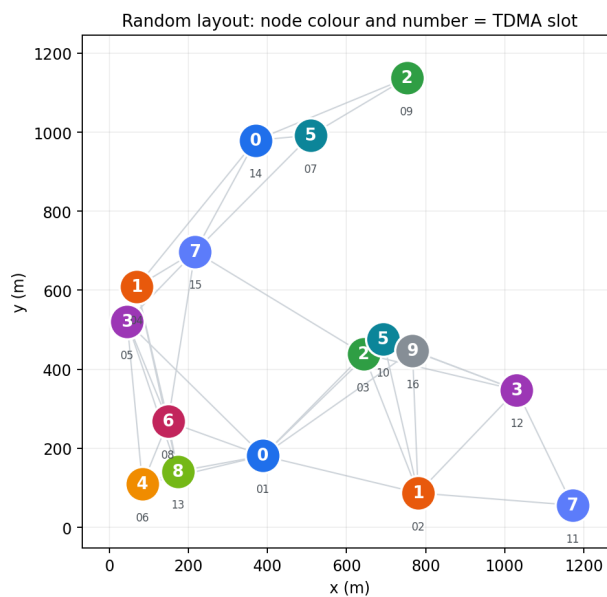


Figure 4. The random 16-node layout, 10 slots.

6. EMANE integration (Part 2)

6.1 Approach

The pipeline is: coordinates, then `schedule_optimizer.py` (Part 1), then `schedule.json`, then `emane/bridge.py`, then EMANE with one emane process per radio. Every radio runs in its own Linux network namespace inside one privileged Docker container; a bridge network carries the EMANE control multicast; the transport creates an `emane0` interface with address `10.100.0.N` for NEM `N`.

6.2 Schedule mapping

- One frame of `N` slots, each 1 ms by default (`slotduration = 1000` microseconds), one frequency (2.4 GHz). Separation is by time, not frequency.
- A node with colour `s` transmits in slot `s`. All nodes sharing that colour are listed together, which is the spatial reuse.
- In every other slot the node listens (rx), so it can decode neighbours. A node never transmits and receives in the same slot.
- NEM identifiers come from the node name (Node_07 is NEM 7).
- The 500 m range is reproduced with pathloss events: 80 dB for linked pairs, 250 dB (unreachable) otherwise, with the PHY in precomputed propagation mode.

6.3 Results

Check	Result
Ping every node pair; reply expected only within 500 m (10 ms slots)	120 / 120 correct
Same check, 1 ms slots, laptop container without realtime scheduling	92 / 120 correct
NEM 1 neighbour table	NEMs 2, 5, 6 only: exactly the radios within 500 m
16 simultaneous flows, this schedule (9 slots)	mean loss 0.8 %, worst flow 4 %
16 simultaneous flows, plain distance-1 colouring (4 slots)	mean loss 43.1 %, worst flow 78 %

The result that matters: the shorter naive schedule loses almost half of the traffic to hidden-terminal collisions, while the distance-2 schedule delivers nearly everything. The loss that remains for the distance-2 schedule comes from the emulation timing, not from collisions.

6.4 Problems met during the first real run

- The PHY profile requires a subid parameter; EMANE aborted without it.
- EEL timestamps reject `-Inf`; changed to `0.0`.
- `emane` takes `--pidfile`; `-p` is the realtime priority.
- The event service resolves `scenario.eel` relative to its working directory.
- The image lacked `python3-networkx`.
- 1 ms slots are not reliable without realtime scheduling; the container run accepts `SLOT_US=10000` for a dependable run. This is documented rather than hidden.

7. Limitations and future work

- The model is a unit-disk graph with a hard 500 m cut-off. Real propagation has fading and capture effects.
- Every radio gets one slot per frame, i.e. equal airtime. Per-node demand would need weighted colouring (several slots for busy nodes).
- Positions are static. With mobility, only the affected neighbourhood should be recoloured instead of replanning everything.
- The clique lower bound is not always tight on large sparse graphs; the report then prints an interval instead of claiming optimality.
- Multi-frequency scheduling (one multiframe per channel) would shorten frames further and is supported by the EMANE format, but is not implemented.
- A faithful 1 ms emulation needs realtime scheduling on a host with spare cores.

8. Reproducing the results

```
pip install -r requirements.txt
python -m pytest -q                                # 29 tests
python schedule_optimizer.py --nodes-file examples/grid_16.json --json-out out.json
python emane/bridge.py out.json --out emane/build    # EMANE files
python emane/slot_sim.py out.json                   # slot-level replay
SLOT_US=10000 sh emane/run_in_docker.sh examples/grid_16.json # EMANE run
BASELINE=1 SLOT_US=10000 sh emane/run_in_docker.sh examples/grid_16.json # naive schedule
```

9. Tools, assistance and references

The brief allows AI assistance provided the candidate can explain and defend every part of the submission. AI tooling was used during development; all design decisions, algorithms and results in this report are understood and can be explained and reproduced from the repository.

References

- EMANE TDMA model guide and wiki (Adjacent Link LLC), schedule XML format and emaneevent-tdmaschedule.
- NetworkX documentation.
- D. Brelaz, New methods to color the vertices of a graph (DSATUR), 1979.
- A. Hertz and D. de Werra, Using tabu search techniques for graph coloring (TabuCol), 1987.
- Literature on distance-2 graph colouring and hidden-terminal avoidance in wireless networks.

Appendix A. Full program output for the 16-node grid

```
=====
                        TDMA TOPOLOGY OPTIMIZATION REPORT
=====
Total Nodes Processed   : 16
Configured Radio Range : 500.0 meters
Radio Links (1-hop)    : 42
Optimized Frame Length  : 9 unique timeslots (Lower is better)
Lower Bound             : 9 (frame length is provably optimal)
=====
NODE -> SLOT ASSIGNMENTS:
Node_01: Slot 0
Node_02: Slot 1
Node_03: Slot 2
Node_04: Slot 0
Node_05: Slot 3
Node_06: Slot 4
Node_07: Slot 5
Node_08: Slot 3
Node_09: Slot 6
Node_10: Slot 7
Node_11: Slot 8
Node_12: Slot 6
Node_13: Slot 0
Node_14: Slot 1
Node_15: Slot 2
Node_16: Slot 0

SLOT -> NODES (spatial reuse):
Slot 0: 4 node(s) transmit together [Node_01, Node_04, Node_13, Node_16]
Slot 1: 2 node(s) transmit together [Node_02, Node_14]
Slot 2: 2 node(s) transmit together [Node_03, Node_15]
Slot 3: 2 node(s) transmit together [Node_05, Node_08]
Slot 4: 1 node(s) transmit together [Node_06]
Slot 5: 1 node(s) transmit together [Node_07]
Slot 6: 2 node(s) transmit together [Node_09, Node_12]
Slot 7: 1 node(s) transmit together [Node_10]
Slot 8: 1 node(s) transmit together [Node_11]

STRUCTURAL TDMA SCHEDULE MATRIX (Slot x Node Boolean Matrix):
Slot \ Node | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16
-----
Slot 00     | 1  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 1
Slot 01     | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0
Slot 02     | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0
Slot 03     | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0
Slot 04     | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0
Slot 05     | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0
Slot 06     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 1  | 0  | 0  | 0  | 0
Slot 07     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0
Slot 08     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0
-----
Execution finalized cleanly. Schedule verified conflict-free.
=====
```